

Linear Programming Course Summary

Generated by Scribe.AI

December 4, 2024

1 Key Terms

- **Slack Variable:** A variable added to a less-than-or-equal-to inequality constraint to transform it into an equality constraint. It represents the difference between the left-hand side and right-hand side of the inequality.
- **Feasible Region:** The set of all points that satisfy all the constraints of a linear programming problem.
- **Vertex:** A point where two or more constraint boundaries intersect in the feasible region of a linear programming problem. In the Simplex method, optimal solutions are found at vertices.
- **Basic Variable:** In the Simplex method, a variable that is expressed in terms of non-basic variables in a dictionary. In a basic feasible solution, basic variables are non-negative.
- **Non-basic Variable:** In the Simplex method, a variable set to zero in a given iteration of the algorithm. The values of the basic variables are determined by the values of the non-basic variables.
- **Pivot Operation:** The process of exchanging a basic and a non-basic variable in the Simplex method, leading to a new basic feasible solution.
- **Dictionary:** A representation of a linear programming problem where basic variables are expressed as functions of non-basic variables. It is used in the Simplex method to iteratively find the optimal solution.
- **Auxiliary Problem:** A modified linear programming problem introduced to find an initial feasible solution when the origin is not feasible in the original problem. It involves adding a new variable to shift the feasible region.
- **Largest-Coefficient Rule:** A pivot rule in the Simplex method that selects the entering variable with the largest coefficient in the objective function row of the dictionary.
- **Largest-Increase Rule:** A pivot rule in the Simplex method that selects the entering variable that results in the largest increase in the objective function value.
- **Klee-Minty Example:** A worst-case example for the Simplex method, demonstrating that the number of iterations can grow exponentially with the problem size.
- **Bland's Rule:** A pivot rule in the Simplex method designed to prevent cycling, ensuring termination.
- **Lexicographic Method:** A method for choosing the leaving variable in the Simplex method that ensures termination by preventing cycling.
- **Randomized Pivot Rule:** A pivot rule that randomly permutes the order of variables before applying another rule, such as Bland's rule.
- **Smoothed Complexity:** A framework for analyzing the complexity of algorithms by considering the average-case performance after small random perturbations to the input.

- **Interior-Point Method:** A class of algorithms for linear programming that traverse the interior of the feasible region, in contrast to the Simplex method which follows the boundary.
- **Primal Problem:** A linear programming problem in its standard maximization form, aiming to maximize a linear objective function subject to linear inequality constraints and non-negativity constraints.
- **Dual Problem:** A linear programming problem derived from the primal problem, typically in minimization form, with constraints and objective function related to the primal problem through matrix transposition and sign changes.
- **Weak Duality:** A theorem stating that for any feasible solutions to both the primal and dual problems, the objective function value of the primal problem is always less than or equal to the objective function value of the dual problem.
- **Strong Duality Theorem:** A theorem stating that if a linear programming problem has an optimal solution, then its dual problem also has an optimal solution, and the optimal objective function values of both problems are equal.
- **Complementary Slackness:** A condition stating that for optimal primal and dual solutions, the product of each primal variable and its corresponding dual slack variable is zero, and the product of each dual variable and its corresponding primal slack variable is zero.
- **Negative Transpose Property:** The property that the dual problem's coefficient matrix is the negative transpose of the primal problem's coefficient matrix, and this relationship is preserved throughout the simplex iterations.
- **Lagrange Multiplier:** A variable introduced in the Lagrangian function to incorporate constraints into an optimization problem.
- **Lagrangian Function:** A function that combines the objective function and constraints of an optimization problem using Lagrange multipliers.
- **Weak Duality Theorem:** A theorem stating that the optimal value of the primal problem is always less than or equal to the optimal value of the dual problem.
- $\text{\textbackslash}\xi(\text{\textbackslash}\mu)\text{\textbackslash}$: The maximum value of the Lagrangian function with respect to x , as a function of the Lagrange multipliers μ .
- $\text{\textbackslash}\xi(\text{\textbackslash}\lambda, \text{\textbackslash}\mu)\text{\textbackslash}$: The maximum value of the Lagrangian function with respect to x , as a function of the Lagrange multipliers λ and μ .
- **Diet Problem:** A classic linear programming problem where the goal is to minimize the cost of a diet while meeting minimum nutritional requirements.
- **Simplex Tableau:** A tabular representation of the simplex method's matrix operations, showing the components needed to update the objective function and basic variables during an iteration.
- **Augmented Matrix:** A matrix formed by appending the identity matrix to the coefficient matrix of a system of linear equations, often used in the simplex method to represent constraints with slack variables.
- **Basic Solution:** A solution to a system of linear equations where the number of non-zero variables equals the number of equations. In the simplex method, this corresponds to a vertex of the feasible region.
- **Non-basic Solution:** A solution to a system of linear equations where the number of non-zero variables is less than the number of equations. In the simplex method, the non-basic variables are set to zero.
- **Entering Variable:** The non-basic variable selected to become a basic variable in the next iteration of the simplex method. The selection is typically based on a rule such as the largest-coefficient rule.

- **Leaving Variable:** The basic variable selected to become a non-basic variable in the next iteration of the simplex method. The selection is determined by the minimum ratio test.
- **Optimal Solution:** A feasible solution that optimizes (maximizes or minimizes) the objective function of a linear programming problem.
- **Optimal Dictionary:** A representation of the optimal solution to a linear programming problem, expressing basic variables in terms of non-basic variables and the objective function value.
- **Reduced Costs:** The values $\tilde{Z}_N = (B^{-1}N)^T C_B - C_N$ representing the change in the objective function value per unit increase in a non-basic variable. A non-negative reduced cost indicates that increasing the corresponding non-basic variable will not improve the objective function.
- **Parametric Analysis:** A technique used to study how changes in the parameters of a linear programming problem (objective function coefficients or constraint values) affect the optimal solution.
- **Sensitivity Analysis:** The study of how changes in the parameters of a linear programming problem affect the optimal solution and the range of parameter values for which the current optimal solution remains optimal.
- **Shadow Price:** The rate of change in the optimal objective function value for a unit change in the right-hand side of a constraint.
- **Basis Matrix (B):** A matrix formed by the columns of the constraint matrix corresponding to the basic variables.
- **Non-basis Matrix (N):** A matrix formed by the columns of the constraint matrix corresponding to the non-basic variables.
- **Dual Simplex Method:** A variant of the simplex method that starts with a dual feasible solution and iteratively improves it until a primal feasible and optimal solution is found.
- **Dual Based Phase I Algorithm:** A method used to find an initial feasible solution for the dual problem when the origin is not dual feasible, often as a precursor to the dual simplex method.
- $\backslash\mathbf{B}$: The basis matrix, a submatrix of the constraint matrix A formed by the columns corresponding to the basic variables.
- $\backslash\mathbf{N}$: The non-basis matrix, a submatrix of the constraint matrix A formed by the columns corresponding to the non-basic variables.
- $\backslash\mathbf{c}_B$: The vector of coefficients in the objective function corresponding to the basic variables.
- $\backslash\mathbf{c}_N$: The vector of coefficients in the objective function corresponding to the non-basic variables.
- $\backslash\mathbf{x}_B$: The vector of basic variables.
- $\backslash\mathbf{x}_N$: The vector of non-basic variables.
- $\backslash\mathbf{z}_N$: The vector of reduced costs, indicating the improvement in the objective function if a non-basic variable enters the basis.
- $\backslash\mathbf{z}_B$: The vector of reduced costs corresponding to the basic variables.
- $\Delta \backslash\mathbf{x}_B$: The change in the basic variables when a non-basic variable enters the basis.

- $\tilde{\mathbf{B}}$: The updated basis matrix after a pivot operation.
- $\mathbf{E}_k$: Elementary matrix representing a single pivot operation.
- $\mathbf{u}_{ij}$: A vector used in updating the basis and non-basis matrices.
- **L² Regression (Least Square)**: Minimizes the sum of squared errors between observed and predicted values.
- **L¹ Regression**: Minimizes the sum of absolute errors between observed and predicted values.
- **L[∞] Regression**: Minimizes the maximum absolute error between observed and predicted values.
- **Linear Separability**: A dataset is linearly separable if there exists a hyperplane that perfectly separates the data points into two classes.
- **Maximum Margin Classifier**: A classifier that maximizes the minimum distance from the hyperplane to the closest data points.
- **Support Vector Machine (SVM)**: A classification algorithm that finds the hyperplane that maximizes the margin between classes.
- **Support Vectors**: The data points closest to the hyperplane in an SVM.
- **Smallest Ball Problem**: Finding the smallest radius ball that contains all given points in $\mathbb{R}^d$.
- **Monthly Demands**: The predicted sales of ice cream for each month of the year, used as input for the production planning problem.
- **Average Demand**: The average monthly sales of ice cream over the entire year, used as a reference point in production planning.
- **Production Level**: The amount of ice cream produced in a given month.
- **Surplus**: The excess amount of ice cream remaining at the end of a month after meeting the demand.
- **Cost of Changing Production**: The cost associated with adjusting the production level from one month to the next.
- **Cost of Storage**: The cost of storing surplus ice cream from one month to the next.
- **Regular Employees**: Employees with a fixed hourly wage, used in workforce planning.
- **Temporary Employees**: Employees hired on a temporary basis, with a higher hourly wage than regular employees.
- **Portfolio Selection**: The problem of choosing the optimal proportions of different investments to maximize return while managing risk.
- **Annual Return**: The return on investment for a given asset over a year.
- **Proportion of Investment**: The fraction of a portfolio allocated to a specific investment.
- **Average Return**: The expected return of an investment, calculated as the average of historical returns.
- **Risk**: The uncertainty or variability associated with the return of an investment.
- **Fluctuation**: The deviation of an investment's return from its average return.

- **Variance:** A measure of the dispersion or spread of an investment's returns around its average return.
- **Covariance:** A measure of the joint variability of two investments' returns.
- **Quadratic Programming:** An optimization problem where the objective function is quadratic and the constraints are linear.
- **Smallest Enclosing Ball:** The problem of finding the smallest ball (or hypersphere) that contains a given set of points.
- **Maximum Weight Matching:** The problem of finding a matching in a graph that maximizes the sum of the weights of the edges in the matching.
- **Machine Scheduling:** The problem of assigning jobs to machines to minimize the total completion time.
- **Knapsack Problem:** The problem of selecting a subset of items with weights and values to maximize the total value while not exceeding a weight limit.
- **Cutting Stock Problem:** The problem of cutting large rolls of material into smaller rolls to minimize waste while satisfying customer orders.
- **Sparse Solutions of Linear System:** The problem of finding a solution to a system of linear equations that has a limited number of non-zero elements.
- **ℓ_1 Norm:** The sum of the absolute values of the elements of a vector.
- **Support:** The set of indices corresponding to non-zero elements in a vector.
- **Farkas Lemma:** A fundamental theorem in linear programming stating that a system of linear inequalities $Ax \leq b$ has no solution if and only if there exists a vector y such that $A^T y = 0$, $y \geq 0$, and $b^T y < 0$.
- **Integer Programming:** A mathematical optimization problem where the objective function and constraints are linear, but the variables are restricted to integer values.
- **Relaxed Problem:** The linear programming problem obtained by removing the integer constraints from an integer programming problem.
- **Branch and Bound:** A method for solving integer programming problems by recursively partitioning the feasible region into smaller subproblems and bounding the optimal solution within each subproblem.
- **Enumeration Tree:** A tree structure used in the Branch and Bound algorithm to represent the partitioning of the feasible region into subproblems.
- **Pruning:** The process of eliminating branches of the enumeration tree in the Branch and Bound algorithm that cannot lead to a better solution than the best integer solution found so far.
- **Gomory Cuts:** Additional linear constraints added to the relaxed problem of an integer program to cut off non-integer solutions while preserving all integer solutions.
- **Network:** A graph consisting of a set of nodes (N) and a set of edges (A).
- **Spanning Tree:** A connected, acyclic subgraph of a network that includes all nodes of the network.
- **Primal Flow:** A flow assignment in a network that satisfies the flow conservation constraints.
- **Dual Flow:** A set of dual variables that satisfy the dual constraints of the network flow problem.
- **Entering Arc:** An arc selected to enter the spanning tree in the network simplex method.
- **Leaving Arc:** An arc selected to leave the spanning tree in the network simplex method.

- **Cycle:** A closed path in a network.
- **Dual Slack:** The difference between the cost of an arc and the difference in dual variables between its endpoints.
- **Two Disconnected Trees:** The result of removing an arc from a spanning tree, resulting in two separate connected components.
- **Root Node:** A designated node in a network, often used as a reference point for algorithms.
- $\backslash \text{mathcal} \backslash \{I \backslash \}$: The tree containing the root node.
- $\backslash \text{mathcal} \backslash \{II \backslash \}$: The tree not containing the root node.
- **Two-Phased Network Simplex Method:** A method that first finds a primal feasible solution and then optimizes it using the primal network simplex method.
- **Parametric Self-Dual Method:** A method that intermingles primal and dual pivots to reduce a perturbation parameter.
- **Transportation Problem:** A network flow problem where the goal is to minimize the cost of transporting goods from multiple sources to multiple destinations, subject to supply and demand constraints. The constraints are typically inequalities, representing upper bounds on supply and lower bounds on demand.
- **Dummy Node:** A node added to a network flow model to handle inequality constraints. It allows for excess supply or unmet demand, transforming inequality constraints into equality constraints.
- **Upper Bounded Transshipment Problem:** A network flow problem where the flow along each arc is bounded above by a specified upper bound, in addition to the usual supply and demand constraints.
- **Shortest Path Problem:** A graph problem where the goal is to find the path of minimum total weight (or cost) between a source node and a destination node in a weighted graph.
- **Bellman's Equation:** A recursive equation used to solve the shortest path problem. It expresses the shortest distance to a node as the minimum of the distances to its neighbors, plus the cost of the edge connecting them.
- **Dijkstra's Algorithm:** A greedy algorithm for finding the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights.
- **Max Flow:** The maximum amount of flow that can be sent from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Cut:** A partition of the nodes in a network into two disjoint sets, one containing the source node and the other containing the sink node.
- **Cut Capacity:** The sum of the capacities of all edges that cross a cut from the source set to the sink set.
- **Max-Flow Min-Cut Theorem:** The theorem stating that the maximum flow in a network is equal to the minimum capacity of any cut in the network.
- **Augmenting Path:** A path in a network from the source to the sink along which flow can be increased.
- **Ford-Fulkerson Algorithm:** An algorithm for finding the maximum flow in a network by iteratively finding and augmenting along augmenting paths.
- $\mathbf{b}_i$: The supply or demand at node i in a network flow problem.

- c_{ij} : The cost of moving a unit of flow along the arc connecting node i and node j in a network flow problem.
- b_j : A value associated with flow constraints at node j in a network flow problem.
- y_i : The market price of a commodity at node i in a network flow problem, used in the economic interpretation of the network simplex method.
- T : The set of arcs in the spanning tree of a network flow problem.
- $\tilde{z}_{ij}$: The adjusted flow along arc ij after incorporating a competitor's strategy in the network simplex method.
- $\tilde{y}_i$: The adjusted price at node i after incorporating a competitor's strategy in the network simplex method.

2 Problem Types

- **Finding an Initial Feasible Solution:** The Simplex method requires an initial feasible solution. If the origin is not feasible, an auxiliary problem is introduced to find a feasible solution to the original problem.
- **Handling Infeasible Origins:** The origin $(0, 0)$ may not satisfy all constraints in a linear programming problem. Techniques are needed to find a feasible starting point for the Simplex method.
- **Optimizing with Slack Variables:** The Simplex method uses slack variables to convert inequality constraints into equalities, facilitating the iterative optimization process.
- **Graphical Representation of Feasible Regions:** Visualizing the feasible region (the set of points satisfying all constraints) helps understand the Simplex method's iterative movement between vertices.
- **Pivot Operations in the Simplex Method:** The Simplex method iteratively improves the solution by exchanging basic and non-basic variables (pivot operations) in the dictionary.
- **Working with Dictionaries of Variables:** The Simplex method uses a dictionary to represent the linear programming problem, where basic variables are expressed in terms of non-basic variables. Each dictionary corresponds to a vertex of the feasible region.
- **Solving Linear Programs with Three Variables:** Extending the Simplex method to problems with more than two variables requires careful consideration of the constraints and iterative optimization.
- **Worst-Case Analysis of the Simplex Method:** The Klee-Minty example demonstrates that the Simplex method can require an exponential number of iterations in the worst case, depending on the pivot rule used.
- **Impact of Pivot Rules on Simplex Method Efficiency:** Different pivot rules (e.g., largest-coefficient, largest-increase) significantly affect the number of iterations required by the Simplex method, with some rules exhibiting exponential worst-case behavior while others perform better in practice.
- **Empirical Analysis of Simplex Method Performance:** Empirical studies show that the number of iterations in the Simplex method tends to grow proportionally to a power of the minimum of the number of constraints and variables, suggesting a polynomial relationship in practice, although not guaranteed theoretically.
- **Theoretical Bounds on Simplex Method Iterations:** Theoretical analysis of randomized pivot rules provides bounds on the expected number of iterations, which are better than exponential but not polynomial.
- **Smoothed Complexity of the Simplex Method:** The smoothed complexity analysis suggests that small random perturbations to the input linear program can significantly improve the performance of the Simplex method, leading to a polynomial number of iterations with high probability.

- **Existence of Polynomial-Time Linear Programming Algorithms:** The lecture notes the existence of polynomial-time algorithms for linear programming, such as interior-point methods, which contrast with the Simplex method's potential for exponential worst-case behavior.
- **Verifying Optimality using Complementary Slackness:** This problem focuses on using the complementary slackness conditions ($x_j z_j = 0$ and $w_i y_i = 0$) to verify whether a given primal and dual feasible solution pair is optimal.
- **Analyzing the Relationship Between Primal and Dual Objective Functions:** This problem explores the relationship between the objective functions of the primal and dual problems, demonstrating their equality at optimality and the inequalities that hold for feasible solutions.
- **Determining the Four Possible Outcomes of Primal and Dual Problems:** This problem involves identifying which of the four possible scenarios (both optimal, primal unbounded/dual infeasible, dual unbounded/primal infeasible, both infeasible) applies to a given primal and dual linear program pair. An example is provided.
- **Proving the Strong Duality Theorem:** This problem involves proving that if the primal problem has an optimal solution, then the dual problem also has an optimal solution, and the optimal objective function values are equal. The proof uses the simplex method and the negative transpose property.
- **Preserving the Negative Transpose Property During Simplex Iterations:** This problem demonstrates that the negative transpose relationship between the primal and dual problems' coefficient matrices is maintained throughout the simplex method iterations.
- **Formulating the Lagrangian for Equality Constraints:** This problem involves constructing the Lagrangian function $L(x, \mu) = f_0(x) - \sum_{j=1}^n \mu_j g_j(x)$ to incorporate equality constraints $g_j(x) = 0$ into the optimization problem.
- **Formulating the Lagrangian for Inequality and Equality Constraints:** This problem involves constructing the Lagrangian function $L(x, \lambda, \mu) = f_0(x) - \sum_{i=1}^m \lambda_i f_i(x) - \sum_{j=1}^n \mu_j g_j(x)$ to incorporate both inequality constraints $f_i(x) \leq 0$ and equality constraints $g_j(x) = 0$ into the optimization problem.
- **Establishing Weak Duality:** This problem involves proving that the optimal value of the primal problem is less than or equal to the optimal value of the dual problem, i.e., $\max_{x \in \mathcal{G}} \{c^T x\} \leq \min_{\lambda \geq 0, \mu} \{L(x^*, \lambda, \mu)\}$.
- **Establishing Strong Duality:** This problem involves proving that under certain conditions, the optimal values of the primal and dual problems are equal, i.e., $x^*(\lambda) = x^*(\lambda)$.
- **Deriving the Dual Problem from the Lagrangian:** This problem involves deriving the dual problem by analyzing the maximum of the Lagrangian function with respect to the primal variables. This involves determining when the maximum is finite and its value.
- **Applying Complementary Slackness:** This problem involves using the complementary slackness condition ($x_i^* \lambda_i = 0$ for all i) to verify the optimality of solutions to the primal and dual problems.
- **Formulating the Diet Problem as a Linear Program:** This problem involves translating the description of a diet problem (minimizing cost while meeting nutritional requirements) into a mathematical linear program with an objective function and constraints.
- **Formulating the Dual of the Diet Problem:** This problem focuses on constructing the dual linear program corresponding to the primal diet problem, often interpreted as a profit maximization problem from selling nutrient pills.
- **Representing the Diet Problem using Tables:** This problem involves representing the parameters and variables of the diet problem in a tabular format to facilitate the formulation of the linear program.

- **Representing Linear Programs in Matrix Notation:** This problem focuses on expressing linear programs using matrices and vectors, facilitating the application of the simplex method.
- **Transforming Linear Programs for Simplex Iterations:** This problem involves converting a linear program into a form suitable for iterative solution using the simplex method, often involving the introduction of slack variables and a modified objective function.
- **Partitioning Variables into Basic and Non-Basic Sets:** This problem deals with classifying variables as basic or non-basic within the simplex method framework, which is crucial for iterative updates.
- **Deriving the Objective Function in Matrix Form for Simplex Iterations:** This problem involves expressing the objective function in terms of basic and non-basic variables using matrix operations, enabling efficient updates during simplex iterations.
- **Updating the Simplex Tableau Using Matrix Operations:** This problem focuses on the iterative updates of the simplex tableau using matrix operations, including inverting the basis matrix and performing matrix multiplications.
- **Illustrating Complementary Slackness with Primal and Dual Variables:** This problem involves demonstrating the relationship between primal and dual variables, specifically the complementary slackness conditions, within the simplex method.
- **Solving a Linear Program Using the Simplex Method in Matrix Form:** This problem involves applying the simplex method in matrix notation to a specific linear program, including identifying entering and leaving variables, updating the tableau, and determining the optimal solution.
- **Solving a Linear Program Graphically:** This problem involves using a graphical method to solve a linear program, identifying the feasible region and the optimal solution point.
- **Determining the range of objective function coefficient changes that preserve optimality:** This problem investigates how much the coefficients in the objective function can change before the optimal solution changes. The analysis uses the reduced costs of the non-basic variables to determine the allowable range of changes.
- **Determining the range of constraint value changes that preserve optimality:** This problem investigates how much the right-hand side values of the constraints can change before the optimal solution changes. The analysis uses the optimal values of the basic variables and the inverse of the basis matrix to determine the allowable range of changes.
- **Deriving the Dual Problem from the Primal Problem:** This problem focuses on deriving the dual linear program from a given primal linear program. The derivation involves techniques such as forming the Lagrangian and manipulating inequalities to obtain the dual objective function and constraints.
- **Handling Unrestricted Variables in Linear Programming:** This problem addresses how to handle linear programs where variables are not restricted to be non-negative. A common approach is to reformulate the problem by splitting each unrestricted variable into the difference of two non-negative variables.
- **Proving Weak Duality:** This problem involves demonstrating that the objective function value of any feasible solution to the primal problem is less than or equal to the objective function value of any feasible solution to the dual problem.
- **Sensitivity Analysis:** Determining how changes in constraint values or objective function coefficients affect the optimal solution and objective function value.
- **Deriving and Utilizing the Dual Problem:** Formulating the dual linear program from the primal problem and using it to verify optimality and perform sensitivity analysis.
- **Analyzing the Simplex Method using Matrix Notation:** Representing the simplex method's iterations and calculations using matrix operations, including the basis and non-basis matrices.

- **Defining Convex Sets and Convex Combinations:** This problem involves defining a convex set as a set where any line segment connecting two points within the set is also contained within the set. It also defines a convex combination as a weighted sum of points where the weights are non-negative and sum to one.
- **Proving Properties of Convex Sets:** This problem focuses on proving that a set is convex if and only if it contains all convex combinations of its points. The proof involves demonstrating this for different numbers of points ($n=2$, $n=3$, $n=4$) and then generalizing the result.
- **Defining and Proving Properties of Convex Hulls:** This problem involves defining the convex hull of a set as the smallest convex set containing the original set. It then proves that the convex hull is equivalent to the set of all convex combinations of finitely many points from the original set.
- **Applying the Caratheodory Theorem:** This problem involves applying the Caratheodory Theorem, which states that any point in the convex hull of a set in $\mathbb{R}^m$ can be expressed as a convex combination of at most $m + 1$ points from the set. The application uses a linear program to demonstrate this.
- **Proving the Separation Theorem:** This problem involves proving the Separation Theorem, which states that two disjoint nonempty polyhedra in $\mathbb{R}^n$ can be separated by two disjoint half-spaces. The proof utilizes Farkas' Lemma.
- **Proving Farkas' Lemma:** This problem focuses on proving Farkas' Lemma, a fundamental result in linear programming that provides a necessary and sufficient condition for the infeasibility of a system of linear inequalities. The proof uses primal and dual linear programs.
- **Regression:** Different methods for regression are presented, focusing on minimizing different error functions (L^2 , L^1 , L^{inf}) to estimate a central tendency from data points.
- **Linear Regression:** Given coefficients $a_1, a_2, \dots, a_n$ and a target value b , find parameters $x_1, x_2, \dots, x_n$ that minimize the error in the equation $b = a_1x_1 + a_2x_2 + \dots + a_nx_n + \text{error}$.
- **Linear Regression with Matrix Notation:** Given data points b_i and coefficients a_{ij} , find $x_1, x_2, \dots, x_n$ such that $b_i = \sum_{j=1}^n a_{ij}x_j + \text{error}_i$, or in matrix form, $b = AX + \text{error}$.
- **Binary Classification:** Find a hyperplane that separates labeled data points (x_i, x'_i) into two classes.
- **Linear Separability:** Determine if there exists a hyperplane that separates a set of labeled data points (x_i, x'_i) into two classes.
- **Maximum Margin Classifier:** Find the hyperplane that maximizes the minimum distance from the hyperplane to the closest data points.
- **Support Vector Machine (SVM):** Find a hyperplane that maximizes the margin between two classes of data points. Identify support vectors as the closest points to the hyperplane.
- **Lagrangian Formulation of SVM:** Formulate the SVM problem using the Lagrangian, resulting in a quadratic programming problem to minimize $\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^N \alpha_i$ subject to constraints.
- **Geometric Interpretation of SVM:** Determine the distance of data points from a hyperplane to understand the margin maximization in SVM. <SLIDE 12, SLIDE 17>
- **SVM Optimization as a QP Problem:** Formulate the SVM problem as a quadratic programming problem, maximizing the margin δ subject to constraints ensuring correct classification. <SLIDE 13, SLIDE 14, SLIDE 19, SLIDE 20>
- **Maximum Inscribed Circle in a Convex Polyhedron:** Find the largest circle that can be inscribed in a convex polyhedron. <SLIDE 15, SLIDE 16, SLIDE 21>

- **Distance of a Point from a Hyperplane:** Calculate the distance (d_i) of a point from a hyperplane using the formula $d_i = \frac{|a_1 s_i + a_2 s_i^2 - b'|}{\sqrt{a_1^2 + a_2^2}}$.
- **Smallest Ball Problem:** Find the smallest radius ball that contains all given points $p_1, \dots, p_n$ in $\mathbb{R}^d$.
- **Production Planning:** Formulating and solving a linear program to optimize ice cream production over a year, minimizing costs associated with changing production levels and storage, given predicted monthly demands.
- **Workforce Planning:** Determining the optimal mix of regular and temporary employees to minimize labor costs while meeting fluctuating monthly labor demands.
- **Portfolio Selection (Markowitz Model):** Optimizing investment portfolios to maximize expected return while minimizing risk, considering historical data and covariances between assets. This involves formulating and solving a quadratic programming problem.
- **Maximum Weight Matching:** Finding a matching in a weighted graph that maximizes the total weight of the selected edges.
- **Machine Scheduling:** Determining the optimal order to process printing jobs on different machines to minimize the total processing time.
- **Knapsack Problem:** Selecting a subset of items with weights and values to maximize the total value while not exceeding a given weight capacity.
- **Cutting Stock Problem:** Determining how to cut large rolls of paper into smaller rolls of various widths to minimize waste while satisfying customer orders.
- **Sparse Solutions of Linear System:** Finding a solution to an underdetermined system of linear equations that has a limited number of non-zero components. This is often formulated as an l_1 norm minimization problem.
- **Determining the existence of a non-negative solution to $Ax = b$:** This problem investigates whether there exists a vector x with non-negative components that satisfies the equation $Ax = b$. Several equivalent conditions are presented using a vector y .
- **Determining the existence of a solution to $Ax \leq b$:** This problem explores whether a solution x exists that satisfies the inequality $Ax \leq b$. Different conditions are presented depending on whether x is restricted to be non-negative.
- **Integer Programming:** This problem type involves finding the optimal solution to a linear program where some or all of the variables are restricted to integer values.
- **Branch-and-Bound:** A method for solving integer programming problems by systematically exploring the solution space through branching and bounding techniques. Depth-first search is discussed as a preferred strategy.
- **Gomory Cuts:** A method for solving integer programming problems by adding constraints (cuts) to the relaxed linear program to eliminate non-integer solutions.
- **Finding a primal feasible flow in a network:** This problem involves finding a flow x such that $\tilde{B}x = \tilde{b}$ and $x_{ij} = 0$ for $(i, j) \notin T$, where T is a spanning tree. Some components of x might be negative, indicating infeasibility.
- **Finding a dual feasible solution in a network:** This problem involves finding dual variables y_i and z_{ij} such that $y_j - y_i + \sum_{ij \in C_{ij}} z_{ij} = c_{ij}$ for all $(i, j) \in A$. Some components of z_{ij} might be positive, indicating infeasibility.

- **Modifying a spanning tree and updating primal and dual flows:** This problem involves selecting an entering arc and a leaving arc to modify the spanning tree, and then updating the primal and dual flows to maintain feasibility and improve the objective function.
- **Selecting an entering arc in the dual network simplex method:** This problem involves selecting an entering arc to reconnect two disjoint trees resulting from removing a leaving arc, while maintaining dual feasibility. The entering arc must bridge the two subtrees in the opposite direction from the leaving arc and have the smallest dual slack.
- **Finding an initial feasible solution using a two-phased network simplex method:** This problem involves modifying the costs to ensure dual feasibility of an initial tree solution, finding a primal feasible solution, and then using the primal network simplex method to find an optimal solution.
- **Finding an initial feasible solution using the primal network simplex method:** This problem involves modifying the supplies to ensure primal feasibility of an initial tree solution, finding a dual feasible solution, and then using the dual network simplex method to find an optimal solution.
- **Applying a parametric self-dual method:** This problem involves interweaving primal and dual pivots to reduce a perturbation parameter μ from ∞ to zero.
- **Modeling Transportation Problems with Inequality Constraints:** This problem involves formulating a linear program to represent a transportation problem where supply at sources may exceed demand at destinations, or vice versa. Dummy nodes and slack variables are introduced to handle the inequalities and ensure a balanced flow.
- **Formulating and Solving Upper Bounded Transshipment Problems:** This problem focuses on network flow problems where the flow along each arc is constrained by an upper bound. Slack variables are used to transform the problem into a standard form solvable by linear programming techniques.
- **Solving the Shortest Path Problem using Linear Programming:** This problem involves formulating the shortest path problem as a linear program, using flow variables to represent paths and constraints to ensure flow conservation and a single path from source to destination.
- **Solving the Shortest Path Problem using Bellman's Equation:** This problem focuses on using Bellman's equation and an iterative approximation method to find the shortest path in a graph. The algorithm iteratively updates distance estimates until convergence.
- **Solving the Shortest Path Problem using Dijkstra's Algorithm:** This problem involves using Dijkstra's algorithm to find the shortest path in a graph. The algorithm maintains a set of processed nodes and iteratively updates shortest distance estimates.
- **Maximum Flow Problem:** Finding the maximum flow from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Max-Flow Min-Cut Theorem:** Proving that the maximum flow in a network is equal to the minimum capacity of any cut separating the source and sink nodes.
- **Linear Programming Formulation of Maximum Flow:** Representing the maximum flow problem as a linear program, with an objective function and constraints that ensure flow conservation and respect edge capacities.

3 Algorithm Solutions

- **Simplex Method:** Iteratively move from one vertex of the feasible region to another by setting non-basic variables to zero and choosing a new set of (non)basic variables to improve the objective function value. This is equivalent to moving from vertex to vertex in the feasible region.

- **Auxiliary Problem:** When the origin is not feasible, introduce a new variable x_0 and modify the constraints to create an auxiliary problem that maximizes $-x_0$. Solve this auxiliary problem using the Simplex method. If the optimal solution has $x_0 = 0$, then the solution is feasible for the original problem.
- **Largest-Coefficient Rule:** Choose the variable with the largest coefficient in the objective function row to enter the basis.
- **Largest-Increase Rule:** Choose the variable whose entrance into the basis brings about the largest increase in the objective function.
- **Bland's Rule:** Choose the variable with the smallest index among those eligible to enter the basis.
- **Randomized Pivot Rule:** Randomly permute the indices of the variables; then use Bland's rule for entering and the lexicographic method for leaving variables.
- **Weak Duality:** $c^T x \leq b^T y$
- **Strong Duality:** If (P) has an optimal solution x^* , then (D) has an optimal solution y^* , and $c^T x^* = b^T y^*$.
- **Complementary Slackness:** Primal feasible x and dual feasible y are optimal if and only if $x_j z_j = 0$ for all j and $w_i y_i = 0$ for all i .
- **Lagrangian Function:** $L(x, \lambda, \mu) = f_0(x) - \sum_{i=1}^m \lambda_i f_i(x) - \sum_{j=1}^n \mu_j g_j(x)$
- **Primal Problem (Diet Problem):** $\text{Minimize } Z(x) = \sum_j c_j x_j \text{ subject to } \sum_j a_{ij} x_j \geq b_i, x_j \geq 0$
- **Dual Problem (Diet Problem):** $\text{Maximize } Z(y) = \sum_i b_i y_i \text{ subject to } \sum_j a_{ij} y_j \leq c_j, y_i \geq 0$
- **Simplex Method in Matrix Form:** The simplex method iteratively improves a feasible solution by exchanging basic and non-basic variables until an optimal solution is found. This involves updating the objective function and constraints using matrix operations, specifically involving the basis matrix B and its inverse B^{-1} , and the non-basic variable matrix N . The objective function is expressed as $\mathcal{Z} = c_B^T B^{-1} b + (c_N^T - c_B^T B^{-1} N) x_N$, where c_B and c_N are the cost vectors for basic and non-basic variables, respectively, and b is the right-hand side vector of the constraints.
- **Sensitivity Analysis of Objective Function Coefficients:** Determine $\Delta C_N = (B^{-1} N)^T \Delta C_B - \Delta C_N$ such that $\tilde{Z}_N^+ \Delta \tilde{Z}_N \geq 0$, where $\tilde{Z}_N^+ = (B^{-1} N)^T C_B - C_N$ is the vector of reduced costs at the optimal solution.
- **Sensitivity Analysis of Constraint Values:** Determine $\Delta X_B = B^{-1} \Delta b$ such that $X_B^+ \Delta X_B \geq 0$, where $X_B^+ = B^{-1} b$ is the vector of basic variable values at the optimal solution.
- **Derivation of the Dual Problem:** The dual problem is derived by taking a linear combination of the primal problem's constraints using Lagrange multipliers. For a primal problem $\max c^T x$ subject to $Ax \leq b$ and $x \geq 0$, the Lagrangian is $\mathcal{L}(x, y) = c^T x - y^T (Ax - b)$, where $y \geq 0$. The dual problem is obtained by minimizing the dual function $g(y) = \inf_{x \geq 0} \mathcal{L}(x, y) = b^T y$ subject to $A^T y \leq c$ and $y \geq 0$.
- **Reformulation of Primal Problem with Unrestricted Variables:** To solve a linear program with unrestricted variables, $\max c^T x$ subject to $Ax \leq b$ and $x \in \mathbb{R}^n$, rewrite $x = x^+ - x^-$ where $x^+, x^- \geq 0$. The problem becomes $\max c^T (x^+ - x^-)$ subject to $A(x^+ - x^-) \leq b$ and $x^+, x^- \geq 0$.
- **Sensitivity Analysis:** This technique analyzes how changes in the objective function coefficients or constraint values affect the optimal solution and objective function value of a linear programming problem. It often involves calculating the range of changes that maintain the current optimal basis.

- **Caratheodory's Theorem:** If $z \in \text{Conv}(S)$, then z can be written as a convex combination of at most $m + 1$ points from S .
- **Farkas' Lemma:** The system $Ax \leq b$ has no solution if and only if there exists a vector $y \geq 0$ such that $y^*TA = 0$ and $y^*Tb < 0$.
- **Separation Theorem:** Let P and $\bar{P}$ be two disjoint nonempty polyhedra in $\mathbb{R}^n$. Then there exist disjoint half-spaces H and $\bar{H}$ such that $P \subset H$ and $\bar{P} \subset \bar{H}$.
- **L^2 Regression (Least Square):** $\min_{x_1, \dots, x_n} \sum_{i=1}^m \left| (b_i - \sum_{j=1}^n a_{ij}x_j) \right|^2$ or $\min \|b - AX\|_2^2$, solved by the normal equation $A^T A \hat{X} = A^T b$, yielding $\hat{X} = (A^T A)^{-1} A^T b$ if $(A^T A)^{-1}$ exists.
- **L^1 Regression:** $\min_{x_1, \dots, x_n} \sum_{i=1}^m |b_i - \sum_{j=1}^n a_{ij}x_j|$ or $\min_X \|b - AX\|_1$, solved by introducing slack variables $\hat{\epsilon}_i$ such that $-\hat{\epsilon}_i \leq b_i - \sum_{j=1}^n a_{ij}x_j \leq \hat{\epsilon}_i$ and minimizing $\sum_{i=1}^m \hat{\epsilon}_i$.
- **L^∞ Regression:** $\min_{x_1, \dots, x_n} \max_{i=1, \dots, m} |b_i - \sum_{j=1}^n a_{ij}x_j|$ or $\min_X \|b - AX\|_\infty$, solved by introducing a slack variable δ such that $-\delta \leq b_i - \sum_{j=1}^n a_{ij}x_j \leq \delta$ for all i and minimizing δ .
- **Lagrangian Formulation of SVM:** $\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^N \alpha_i$ subject to $\sum_{i=1}^N \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$, for all i .
- **Smallest Ball Problem:** Given points $p_1, \dots, p_n \in \mathbb{R}^d$, find a ball of the smallest radius that contains all the points.
- **Ice Cream Production Planning:** $\min \sum_{i=1}^{12} (Y_i + Z_i) + 20 \sum_{i=1}^{12} S_i$ s.t. $X_i + S_{i-1} - S_i = d_i$, $X_i - X_{i-1} = Y_i - Z_i$, $X_0 = 0$, $S_0 = 0$, $S_{12} = 0$, $X_i, Y_i, Z_i, S_i \geq 0$
- **Workforce Planning:** $\min \sum_{i=1}^{12} Y_i + (17.5 \text{ times } 12) X$ s.t. $a_i - X \leq Y_i$, $0 \leq Y_i$, $X \geq 0$
- **Portfolio Selection (Markowitz Model):** $\max \mu \sum_{j=1}^n x_j \overline{r_j} - \frac{\epsilon}{2} \sum_{i=1}^n \sum_{j=1}^n x_i x_j \sigma_{ij}$ s.t. $\sum_{j=1}^n x_j = 1$, $x_j \geq 0$
- **Maximum Weight Matching:** $\max \sum_{e \in E} w_e x_e$ s.t. $\sum_{e \in \delta(v)} x_e = 1$ for all vertices v , $x_e \in \{0, 1\}$ for all edges e
- **Machine Scheduling:** $\min \pi_i$ s.t. $\sum_{i \in M} x_{ij} = 1$ for $j \in J$, $\sum_{j \in J} d_{ij} x_{ij} \leq \pi_i$ for $i \in M$, $x_{ij} \in \{0, 1\}$ for all i, j
- **Knapsack Problem:** maximize $\sum_{j=1}^n c_j x_j$ subject to $\sum_{j=1}^n a_j x_j \leq b$, $x_j \in \{0, 1\}$, $j = 1, 2, \dots, n$
- **Cutting Stock Problem:** $\min \sum_{i=1}^{12} x_i$ subject to constraints based on cutting patterns P_i and demand for different widths.
- **Sparse Solutions of Linear System:** $\min \|x\|_1$ s.t. $Ax = b$
- **Branch and Bound:** This algorithm solves integer programming problems by recursively partitioning the feasible region into subproblems, solving the relaxed linear program for each subproblem, and pruning branches that cannot yield a better solution than the best integer solution found so far. A depth-first search strategy is often employed to find integer feasible solutions early, enabling pruning and improving efficiency.

- **Gomory Cuts:** This algorithm adds new constraints (cuts) to the relaxed linear program to eliminate non-integer solutions. The cuts are derived from the fractional parts of the basic variables in the optimal dictionary of the relaxed problem. The process is iterative, adding cuts until an integer solution is found.
- **Network Flow Problem Formulation:** The network flow problem is formulated as a linear program with the objective function $\min \sum_{(i,j) \in A} c_{ij} x_{ij}$ subject to flow conservation constraints $\sum_{i \in X} x_{ik} - \sum_{j \in X} x_{kj} = -b_k$ at each node k and non-negativity constraints $x_{ij} \geq 0$.
- **Primal Simplex Method for Network Flows:** The primal simplex method is adapted for network flows by iteratively selecting an entering variable (arc with the most negative reduced cost), identifying a leaving variable (arc in the cycle formed by the entering variable and the spanning tree), updating the spanning tree, and updating primal and dual variables until all dual slacks are non-negative.
- **Finding a Spanning Tree Basis:** A square submatrix of the modified incidence matrix $\tilde{A}$ (incidence matrix with a row removed corresponding to a root node) is a basis if and only if the arcs corresponding to its columns form a spanning tree.
- **Dual Variable Calculation from Spanning Tree:** Given a spanning tree, dual variables are computed iteratively starting from a root node using the equation $y_j - y_i = c_{ij}$ for arcs (i, j) in the spanning tree. Dual slacks for arcs not in the spanning tree are then computed using $z_{ij} = y_j - y_i + c_{ij}$.
- **Primal Network Simplex:** The algorithm iteratively improves a primal feasible solution by selecting an entering arc with $z_{ij} < 0$, finding a cycle including the entering arc and a leaving arc (the arc with the smallest flow in the reverse direction of the entering arc in the cycle), and updating primal and dual flows along the cycle.
- **Dual Network Simplex:** The algorithm iteratively improves a dual feasible solution by selecting a leaving arc with $x_{ij} < 0$, finding two disjoint trees resulting from removing the leaving arc, selecting an entering arc (the arc with the smallest dual slack bridging the two trees in the opposite direction of the leaving arc), and updating primal and dual flows.
- **Two-Phased Network Simplex Method:** This method first finds a primal feasible solution by temporarily altering costs to ensure dual feasibility of an initial tree solution, then uses the primal network simplex method to find an optimal solution. Alternatively, it can find a dual feasible solution by temporarily altering supplies to ensure primal feasibility of an initial tree solution, then uses the dual network simplex method to find an optimal solution.
- **Parametric Self-Dual Network Simplex Method:** This method intermingles primal and dual pivots to reduce a perturbation parameter μ from ∞ to zero, finding an optimal solution.
- **Bellman's Equation:** $v_i = \min_j \{c_{ij} + v_j\}$, $v_r = 0$
- **Ford-Fulkerson Algorithm (Augmented Path):** This algorithm iteratively finds augmenting paths in a network flow graph and updates the flow along these paths to increase the total flow from the source to the sink until no more augmenting paths can be found. The algorithm terminates when all labeled nodes are scanned.
- **Max-Flow Min-Cut Theorem:** The maximum flow from the source to the sink in a network is equal to the minimum capacity of any cut in the network.
- **Linear Programming Formulation of Max Flow:** The maximum flow problem can be formulated as a linear program with an objective function to maximize the flow from source to sink, subject to constraints ensuring flow conservation at each node and respecting edge capacities $0 \leq x_{ij} \leq u_{ij}$. A fictitious arc from sink to source with infinite capacity is often added.